WEBTECH NETWORK

Criando APIs com Spring Boot

Com Artur Bomtempo Colen

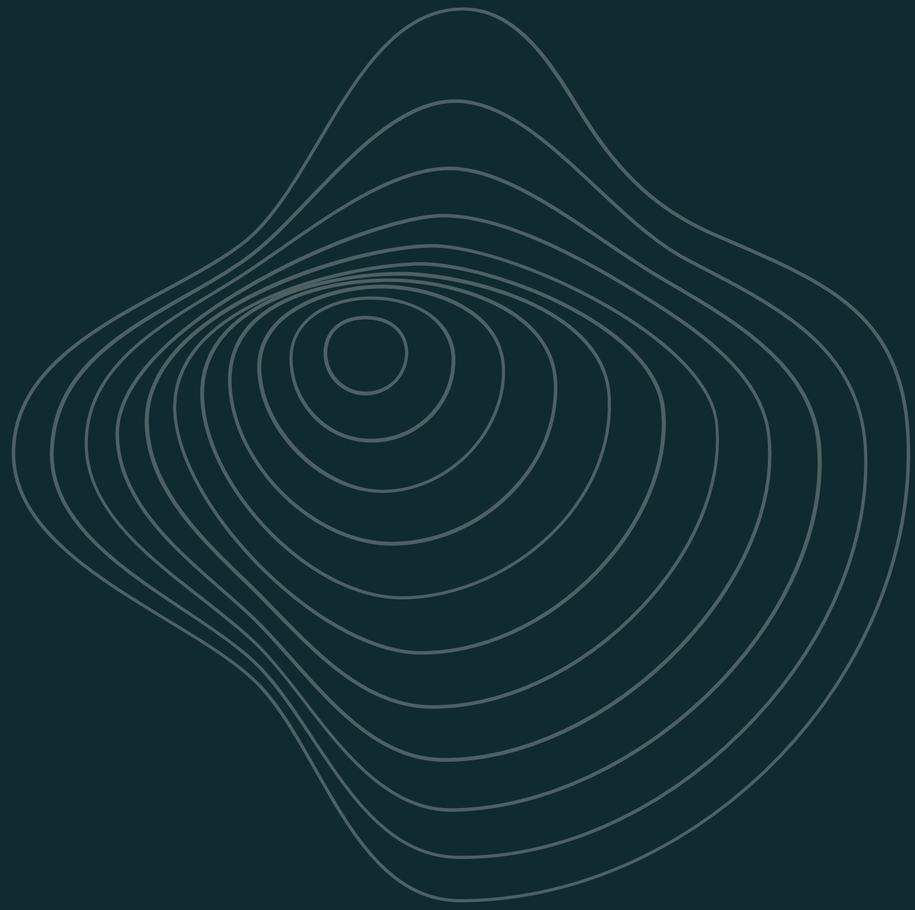
Sobre mim

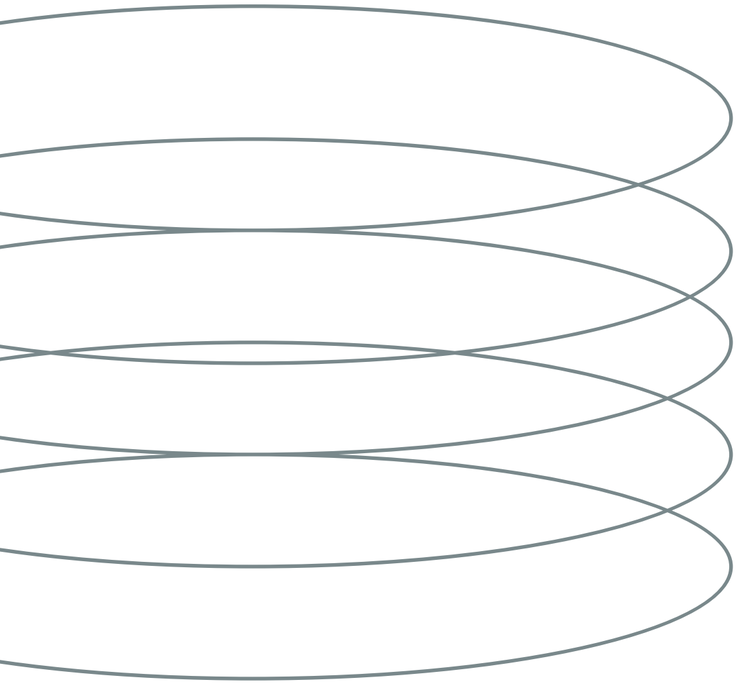
- **Chapter Lead** no WebTech Network
- **Desenvolvedor de Software** na dti digital
- Graduando em **Engenharia de Software** na PUC Minas
- **Técnico em Informática** pelo Colégio Cotemig
- Compartilho **conhecimento** e conteúdo tech

O que vamos construir

API REST com Spring Boot

- Arquitetura em camadas
- CRUD com PostgreSQL
- Integração com React
- Boas práticas de mercado





Sobre o projeto

O **SetupHub** é uma aplicação onde usuários podem cadastrar setups e gerenciar periféricos e equipamentos.

Começando pelo começo

Como aplicações se comunicam?

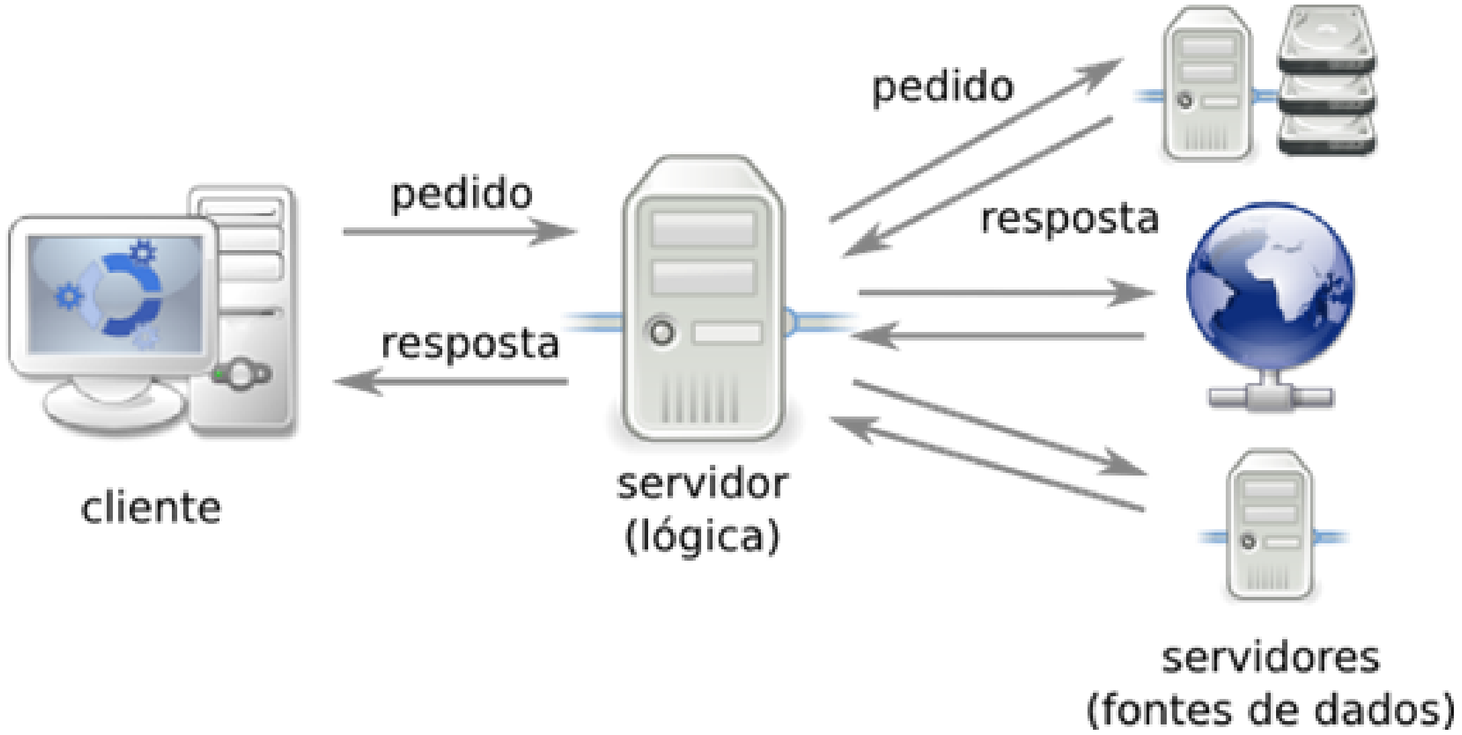
- Front-end e back-end
- Cliente e servidor
- Requisições HTTP
- APIs REST

Arquitetura Cliente-Servidor

Como aplicações modernas funcionam

Cliente → faz requisições

Servidor → processa e responde



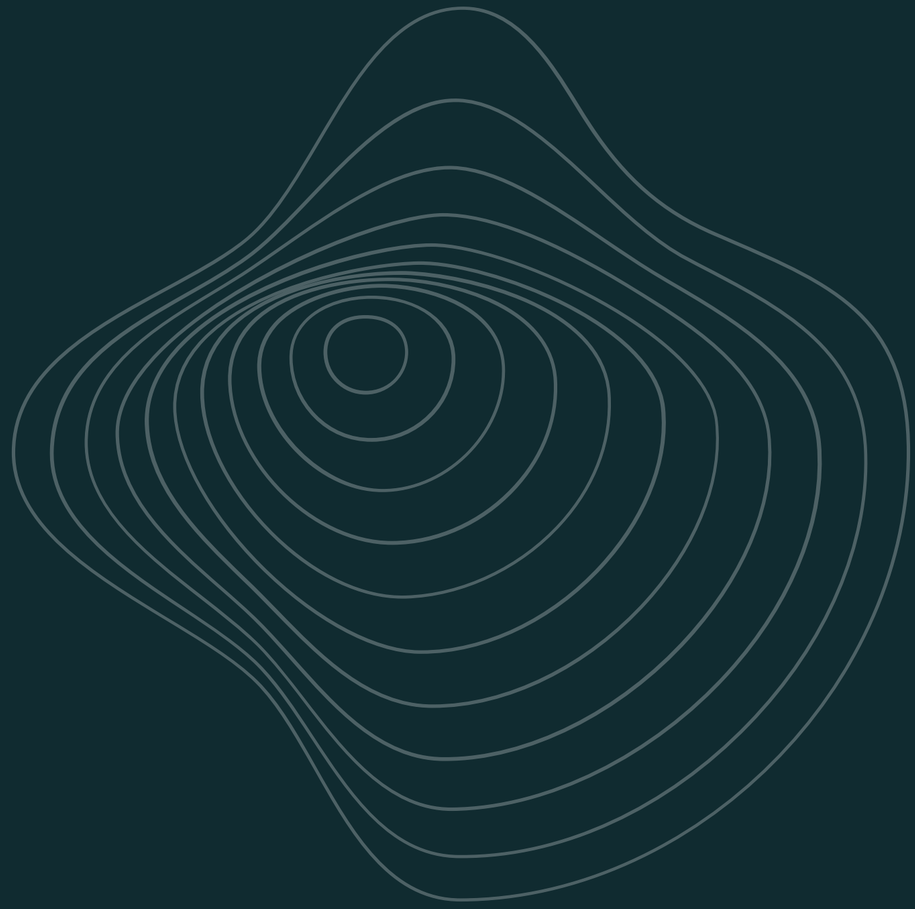
O que é uma API?

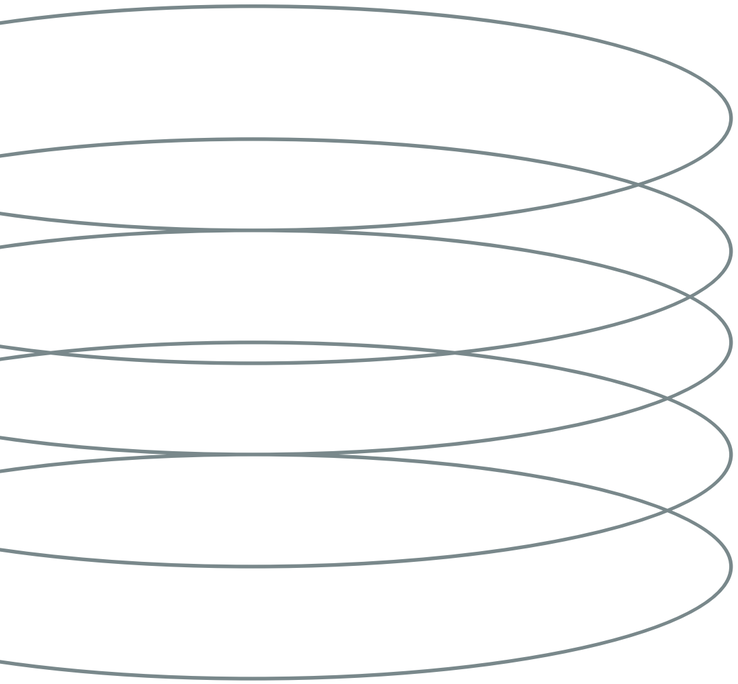
API = Application Programming Interface

- Ponte de comunicação entre sistemas
- Recebe requisições
- Processa dados
- Retorna respostas

Exemplo de API no dia a dia

- Instagram
- Spotify
- Netflix
- iFood





O que é uma API REST?

REST é um padrão utilizado para construção de APIs modernas, baseado em requisições HTTP. Ele organiza a comunicação da aplicação em recursos e endpoints, permitindo operações como criação, consulta, atualização e remoção de dados utilizando **JSON**.

Métodos HTTP

Método	Ação
GET	Buscar
POST	Criar
PUT	Atualizar
DELETE	Remover

Exemplo de API REST

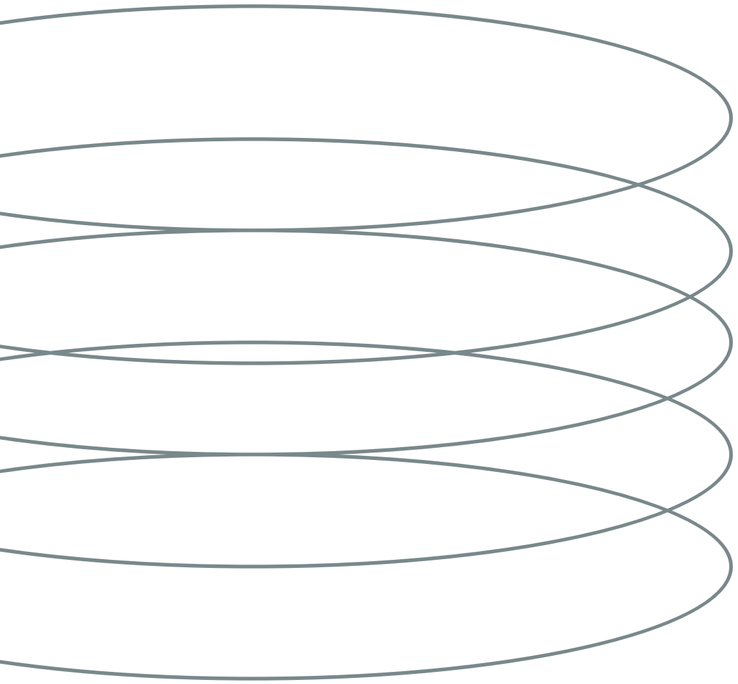
GET /setups

POST /setups

PUT /setups/{id}

DELETE /setups/{id}

Cada rota representa uma ação da aplicação



Como uma requisição funciona

React Frontend

↓

HTTP Request

↓

Spring Boot API

↓

PostgreSQL

E volta: JSON Response

O que é um HTTP Client?

Aplicações utilizadas para testar e consumir APIs através de requisições HTTP

- Envio de requisições
- Visualização de respostas
- Teste de endpoints
- Integração entre sistemas

Reqly

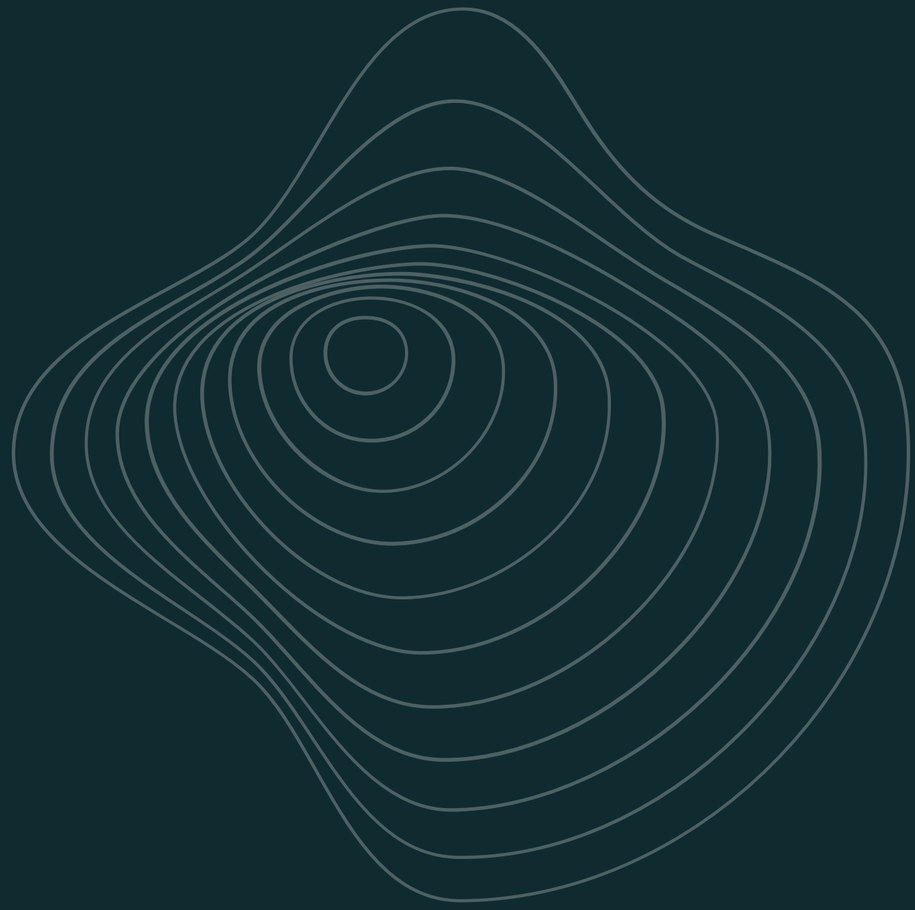
O Reqly é um HTTP Client desenvolvido para simplificar testes e consumo de APIs REST, permitindo realizar requisições HTTP, visualizar respostas em JSON e testar endpoints de forma rápida e prática.

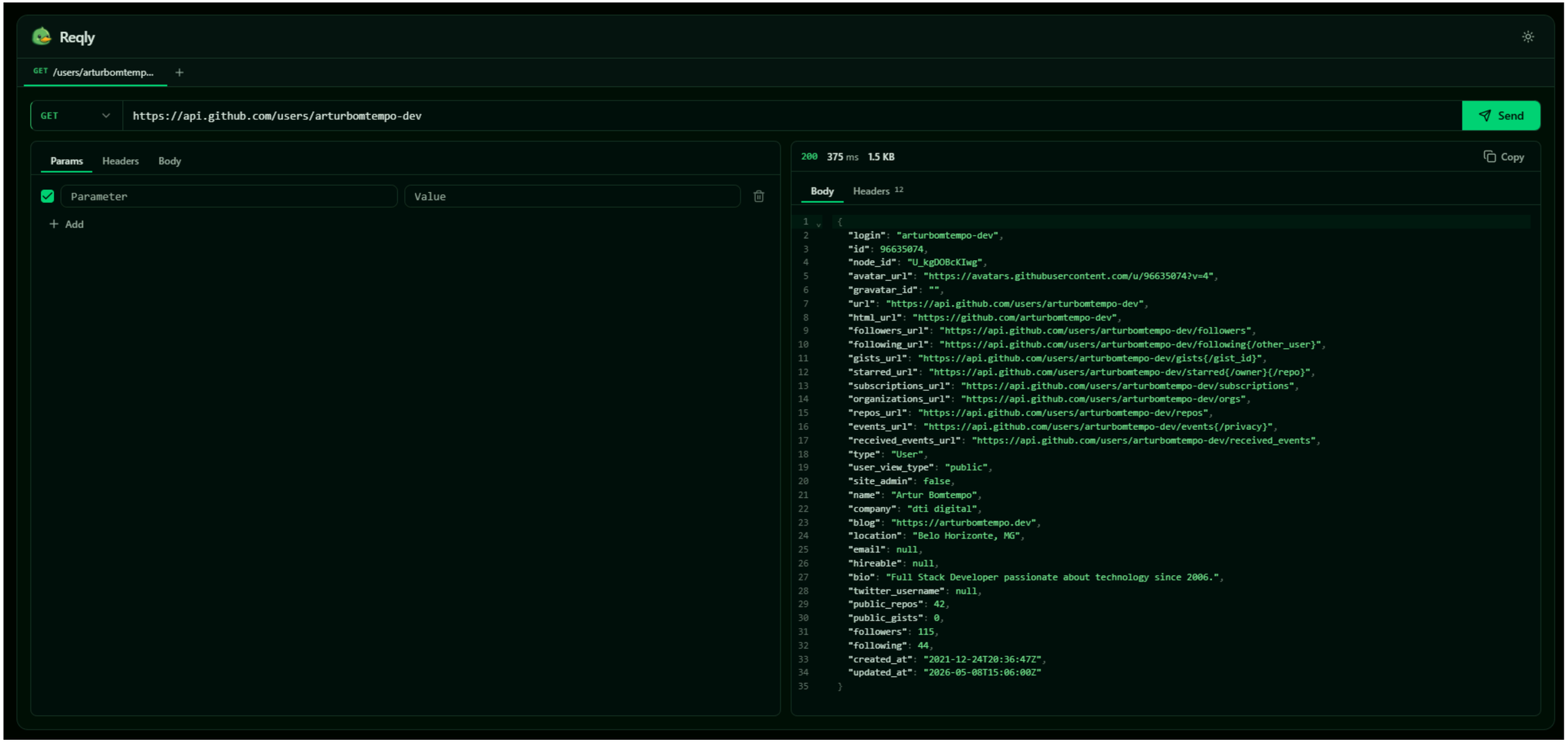
Vamos praticar

Vamos consumir uma API real utilizando HTTP

API pública do GitHub:

https://api.github.com/users/octocat




Reqly
GET /users/arturbomtemp...
GET
https://api.github.com/users/arturbomtempo-dev
Send
Params
Headers
Body
Parameter
Value
Add
200 375 ms 1.5 KB
Copy
Body
Headers 12
{
"login": "arturbomtempo-dev",
"id": 96635074,
"node_id": "U_kgDOBcKIwg",
"avatar_url": "https://avatars.githubusercontent.com/u/96635074?v=4",
"gravatar_id": "",
"url": "https://api.github.com/users/arturbomtempo-dev",
"html_url": "https://github.com/arturbomtempo-dev",
"followers_url": "https://api.github.com/users/arturbomtempo-dev/followers",
"following_url": "https://api.github.com/users/arturbomtempo-dev/following{/other_user}",
"gists_url": "https://api.github.com/users/arturbomtempo-dev/gists{/gist_id}",
"starred_url": "https://api.github.com/users/arturbomtempo-dev/starred{/owner}{/repo}",
"subscriptions_url": "https://api.github.com/users/arturbomtempo-dev/subscriptions",
"organizations_url": "https://api.github.com/users/arturbomtempo-dev/orgs",
"repos_url": "https://api.github.com/users/arturbomtempo-dev/repos",
"events_url": "https://api.github.com/users/arturbomtempo-dev/events{/privacy}",
"received_events_url": "https://api.github.com/users/arturbomtempo-dev/received_events",
"type": "User",
"user_view_type": "public",
"site_admin": false,
"name": "Artur Bomtempo",
"company": "dti digital",
"blog": "https://arturbomtempo.dev",
"location": "Belo Horizonte, MG",
"email": null,
"hireable": null,
"bio": "Full Stack Developer passionate about technology since 2006.",
"twitter_username": null,
"public_repos": 42,
"public_gists": 0,
"followers": 115,
"following": 44,
"created_at": "2021-12-24T20:36:47Z",
"updated_at": "2026-05-08T15:06:00Z"
}


Spring Boot

O que é Spring Boot?

Framework Java para desenvolvimento de APIs

- Criação rápida de aplicações
- Integração com banco de dados
- APIs REST
- Ecossistema Spring

Por que Spring Boot?

- Muito utilizado no mercado
- Alta produtividade
- Organização profissional
- Ecossistema robusto

Evolução do ecossistema Spring

2002

Criação do Spring Framework

2004

Lançamento oficial do Spring

2014

Surgimento do Spring Boot

2018

Spring Boot 2

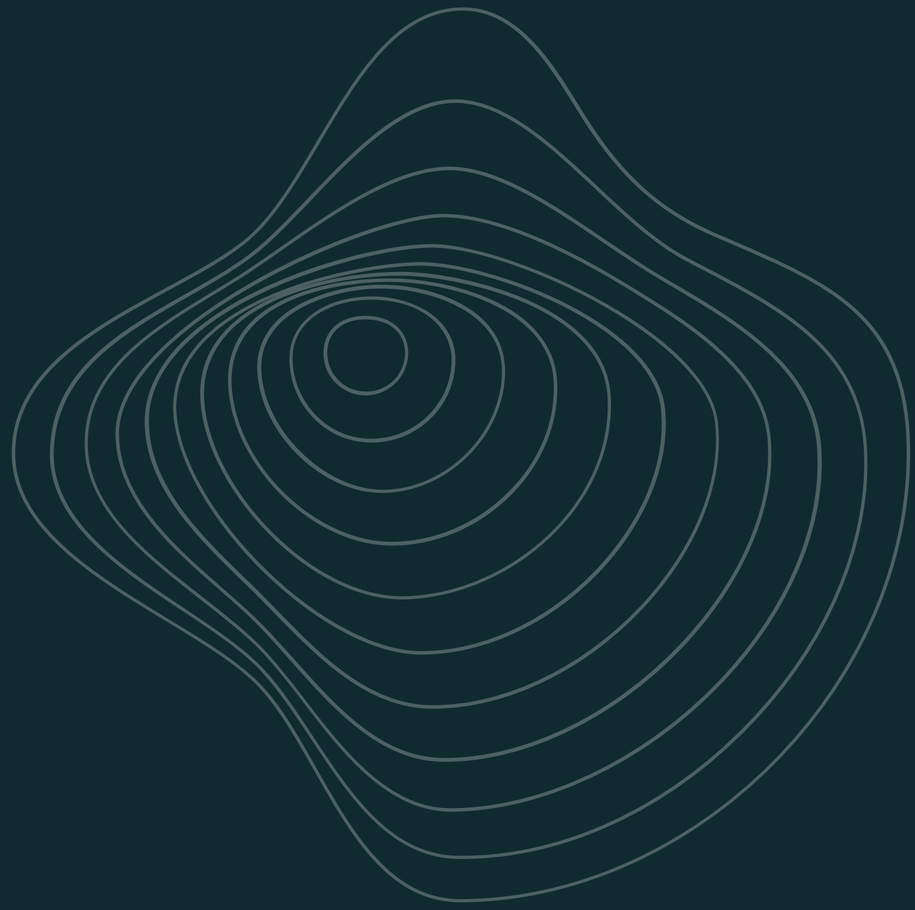
2022-Hoje

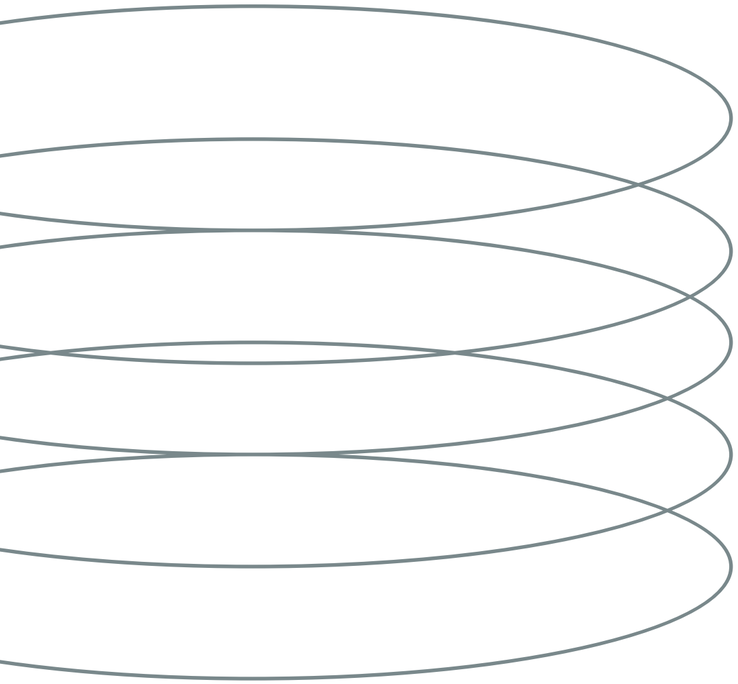
Spring Boot 3 e Java moderno

Ecossistema Spring

Um dos maiores ecossistemas Java do mercado

- Spring Framework
- Spring Boot
- Spring Data JPA
- Spring Security
- Spring Cloud





Annotations no Spring

Annotations são metadados que ajudam o Spring a entender o papel de cada classe e automatizar comportamentos da aplicação.

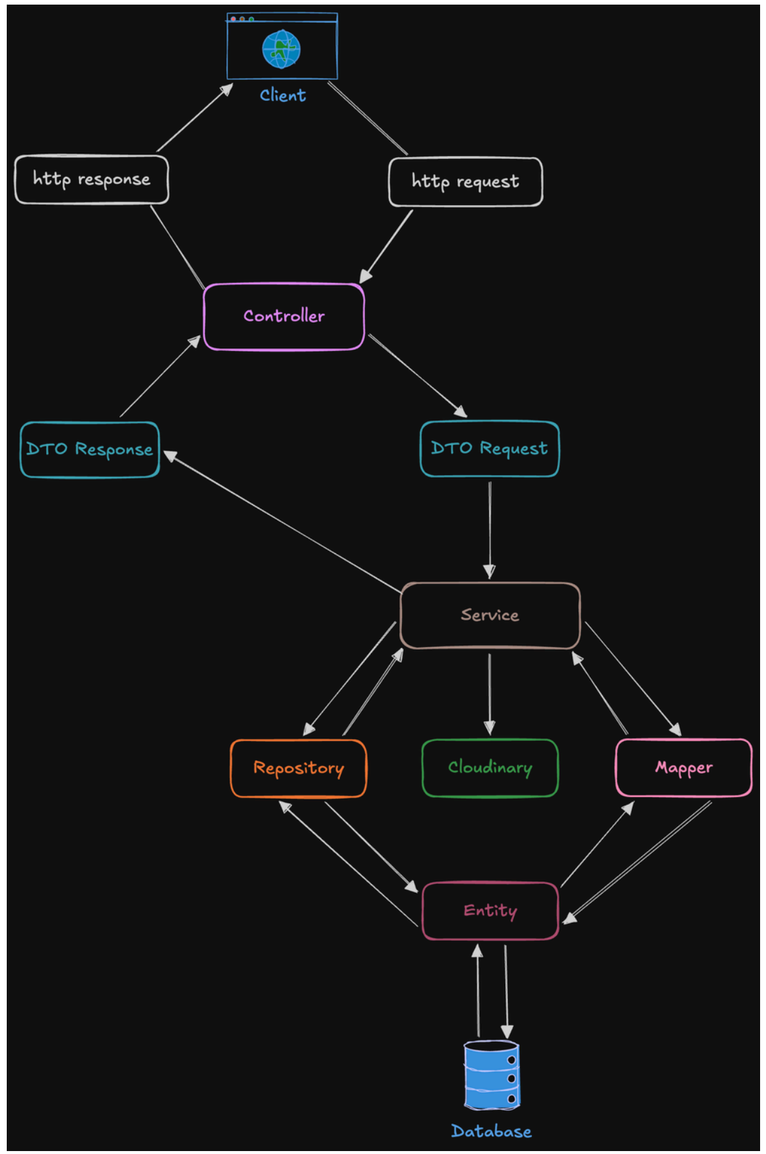
Exemplos:
@RestController
@Service
@Repository
@Entity

Arquitetura em camadas

Separação de responsabilidades da aplicação

- **Controller** → entrada HTTP
- **Service** → regras de negócio
- **Repository** → acesso ao banco
- **Entity** → representação dos dados

Arquitetura em camadas



Containerização

O que é containerização?

Containerização permite executar aplicações de forma isolada e padronizada em qualquer ambiente.

- Ambiente consistente
- Facilidade de configuração
- Banco de dados isolado
- Execução simplificada

Podman

Engine de containers compatível com Docker

- Execução de containers
- Ambiente isolado
- PostgreSQL configurado rapidamente
- Muito utilizado em ambientes modernos

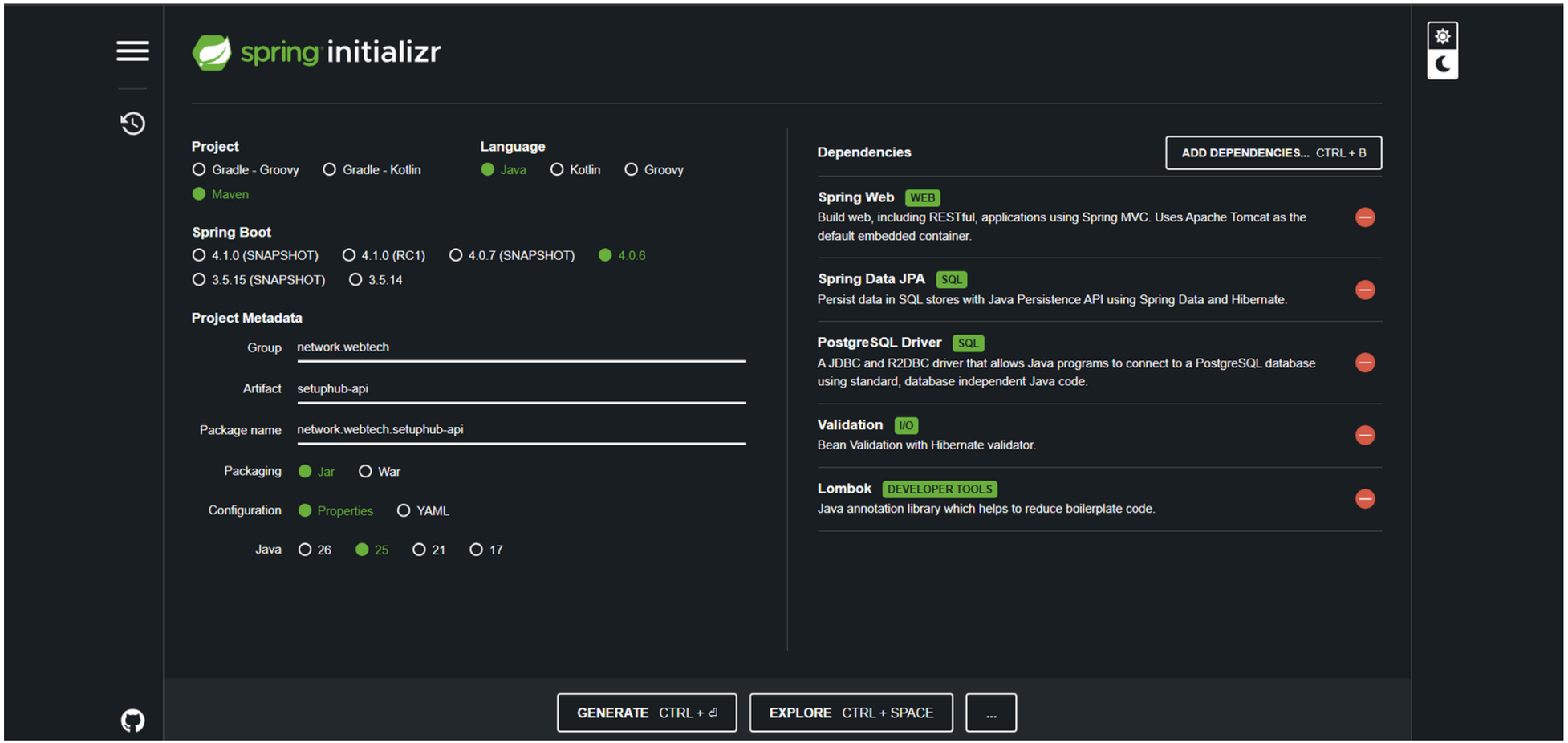
Mão na massa

Configurando o projeto

Acesse: Spring Initializr

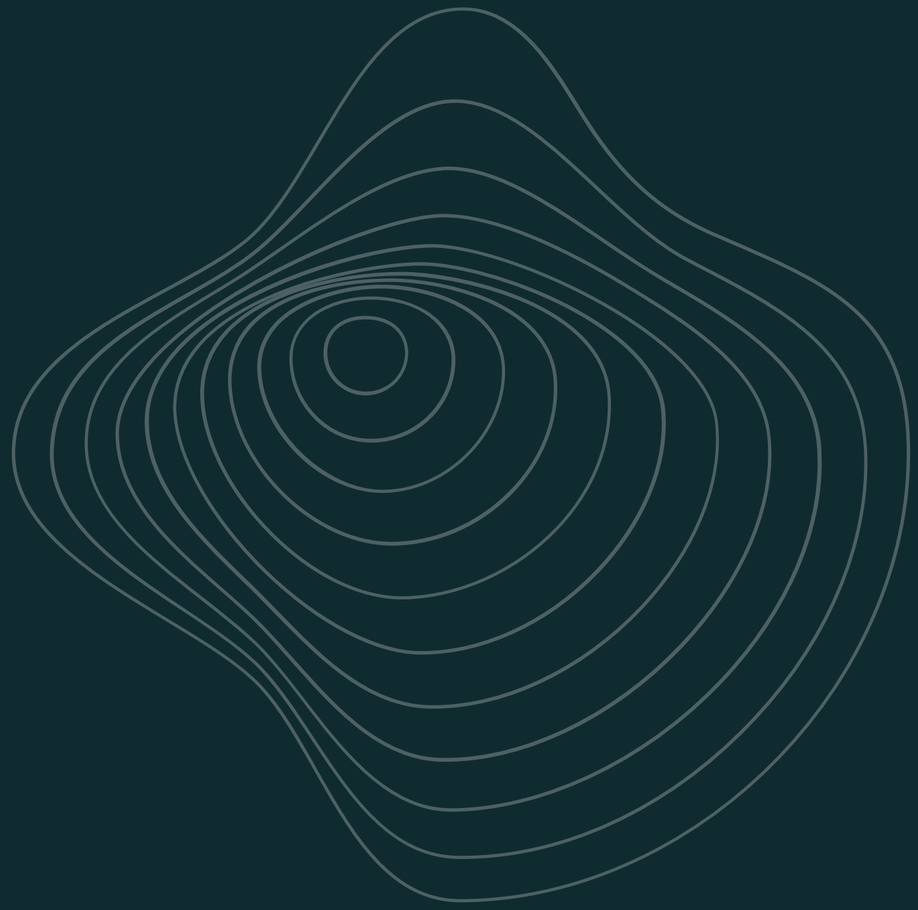
- **Project:** Maven
- **Language:** Java
- **Spring Boot:** 4.0.6 (mais atual em Maio/2025)
- **Group:** network.webtech
- **Artifact:** setuphub-api
- **Packaging:** Jar
- **Java:** 21

Dependências



Dependências adicionais

Algumas dependências serão adicionadas manualmente através do Maven Repository



- **MapStruct** → mapeamento entre DTOs e Entities
- **MapStruct Processor** → geração automática dos mappers
- **Spring Dotenv** → leitura de variáveis do arquivo .env

Configurando o ambiente

Inicializando o Podman

1. Iniciar a máquina do Podman

podman machine start

2. Criar container PostgreSQL

podman run -d --name setuphub-db -e POSTGRES_DB=setuphub-db -e POSTGRES_USER=admin -e POSTGRES_PASSWORD=admin -p 5432:5432 -v pg-data:/var/lib/postgresql/data postgres:17

Verificar conexão

Primeira execução do projeto

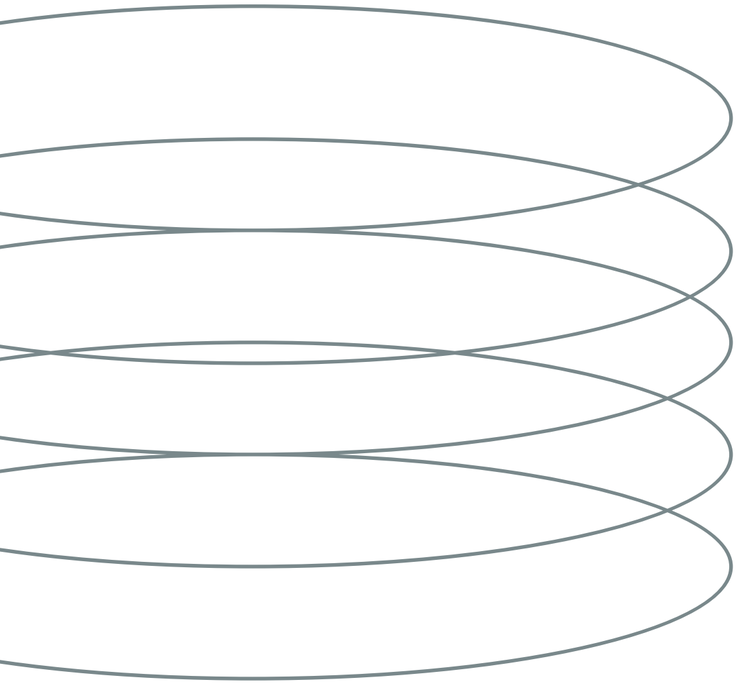
3. Verificar containers

podman ps

4. Rodar Spring Boot

mvn spring-boot:run

Vai falhar. Precisamos configurar o application.properties.

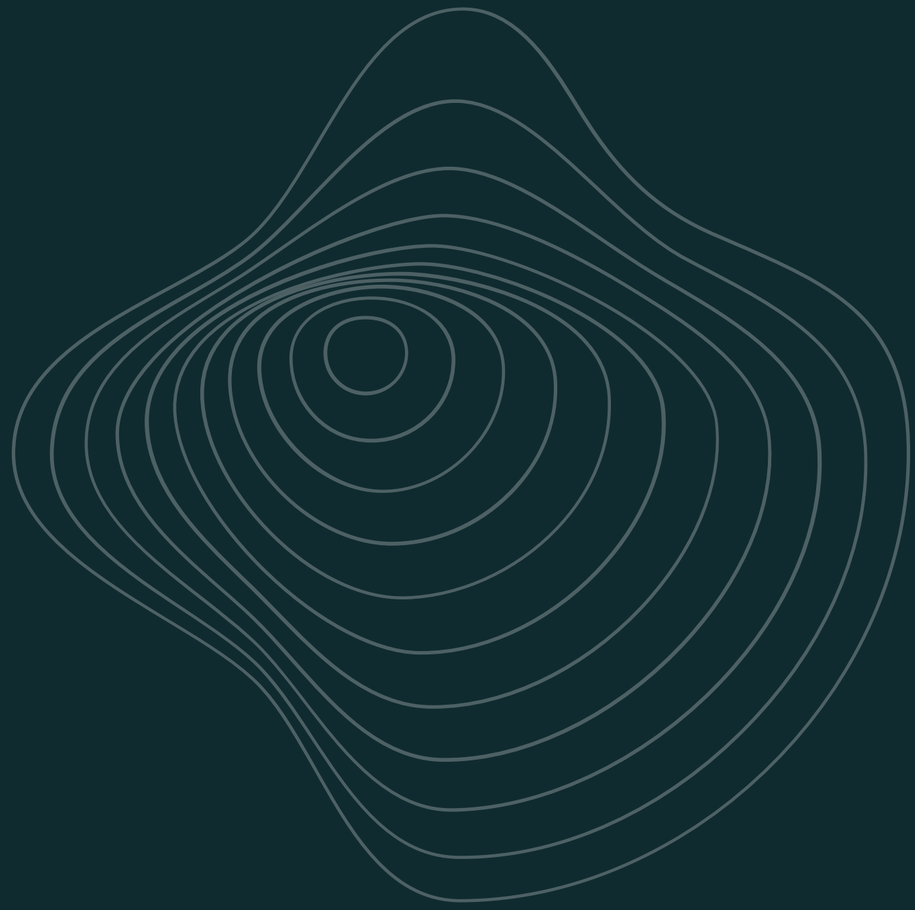


Configurando o application.properties

O **application.properties** é o arquivo responsável pelas configurações do projeto no Spring Boot, como porta da aplicação, conexão com banco de dados e comportamento do JPA.

Primeiro endpoint da API

Criando nosso primeiro controller



Criar a pasta:

controller/

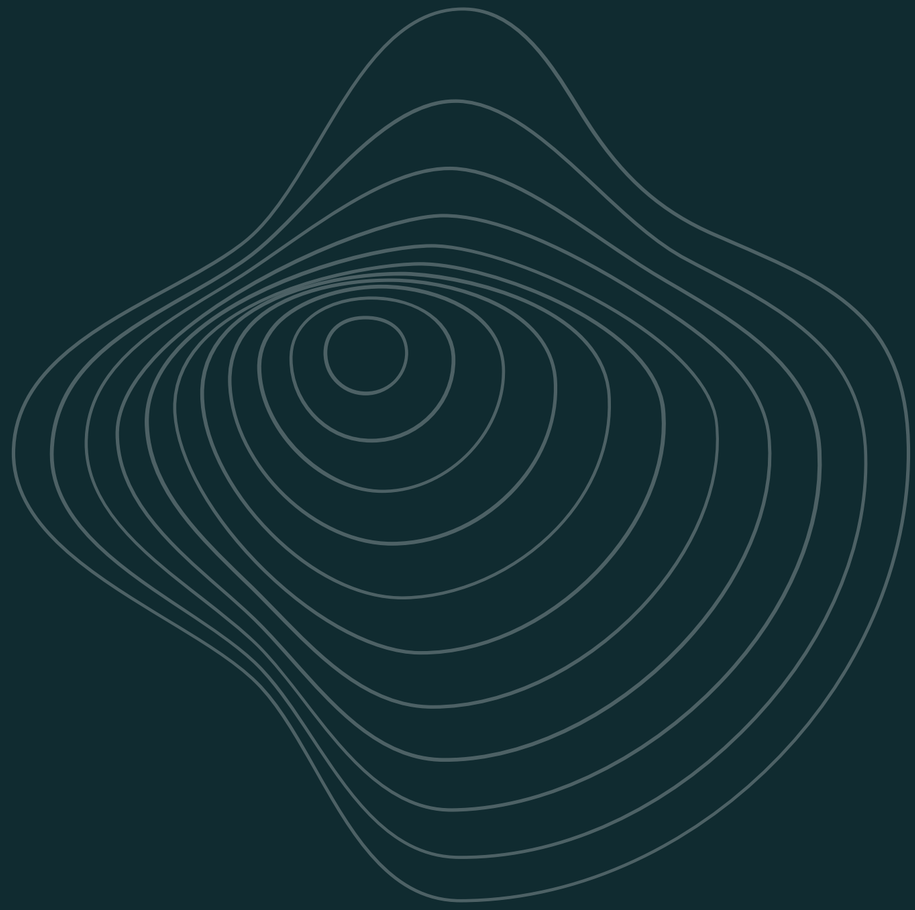
E adicionar a classe:

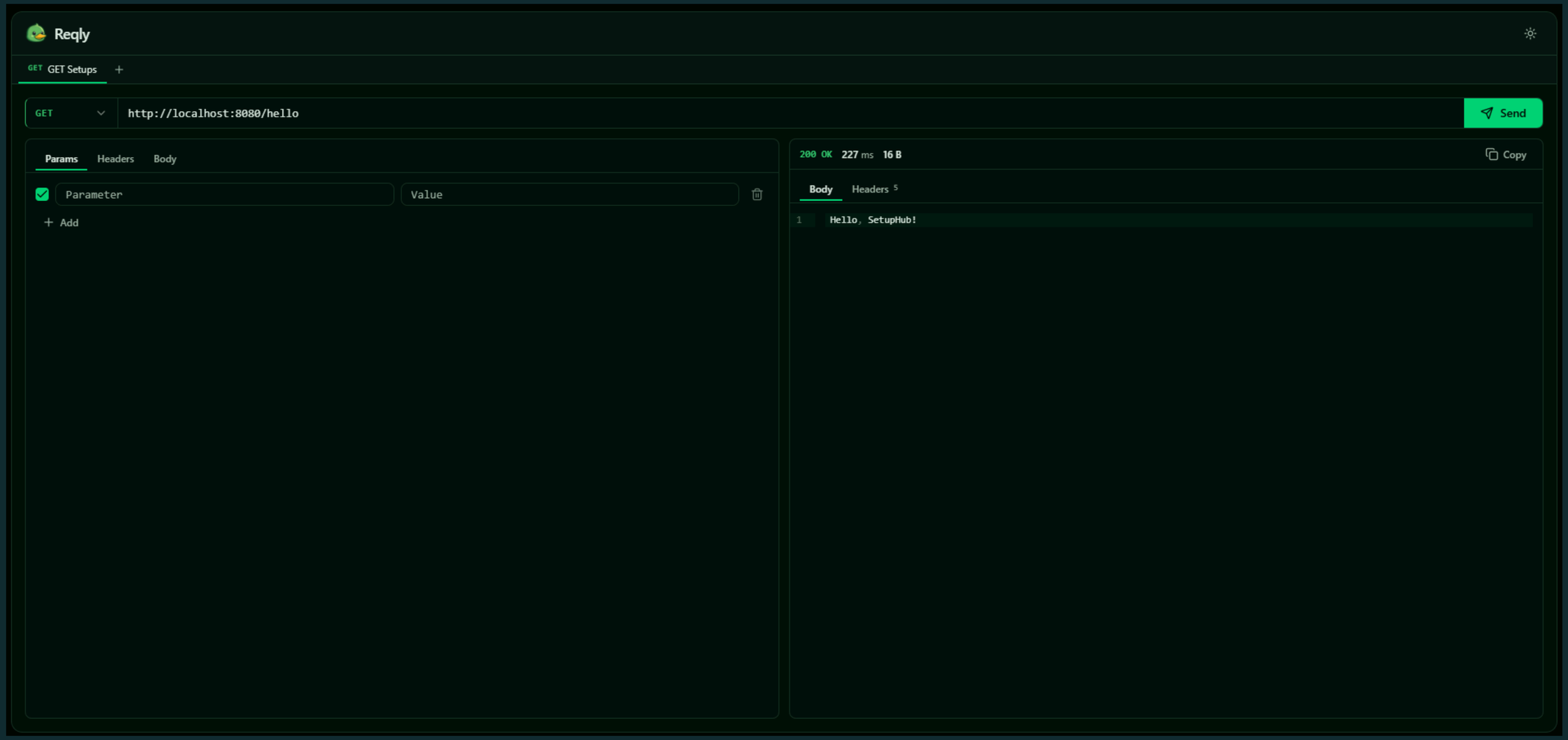
HelloController.java

HelloController.java

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello, SetupHub!";
    }
}
```

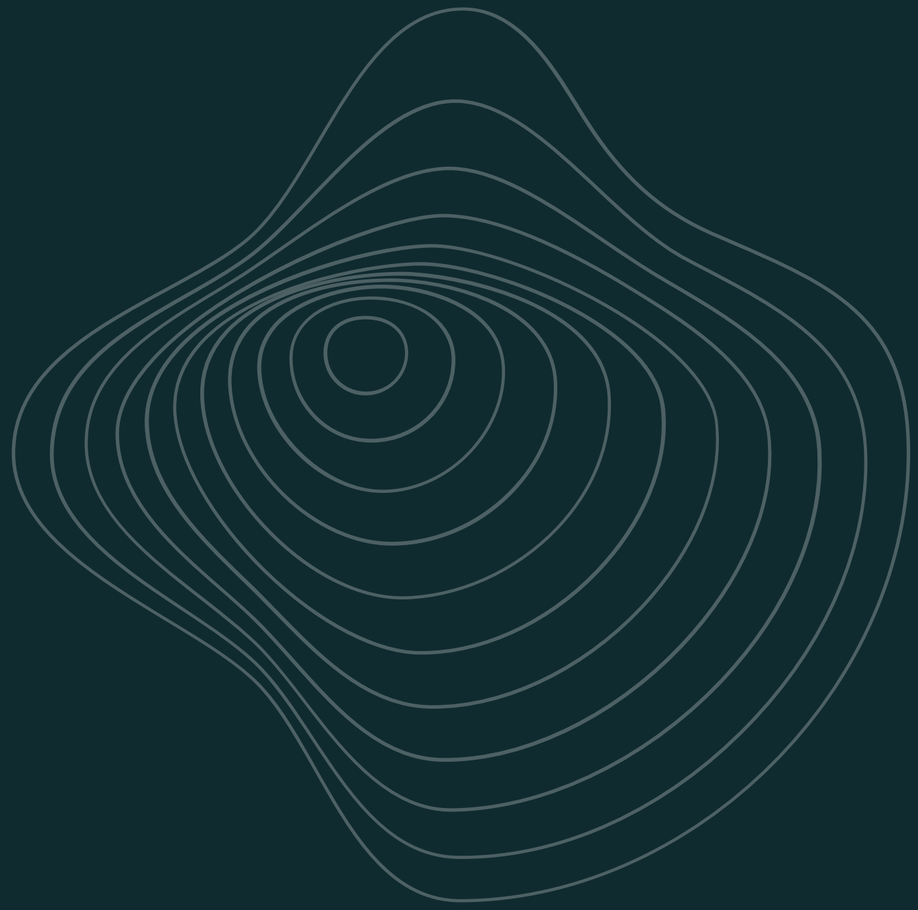


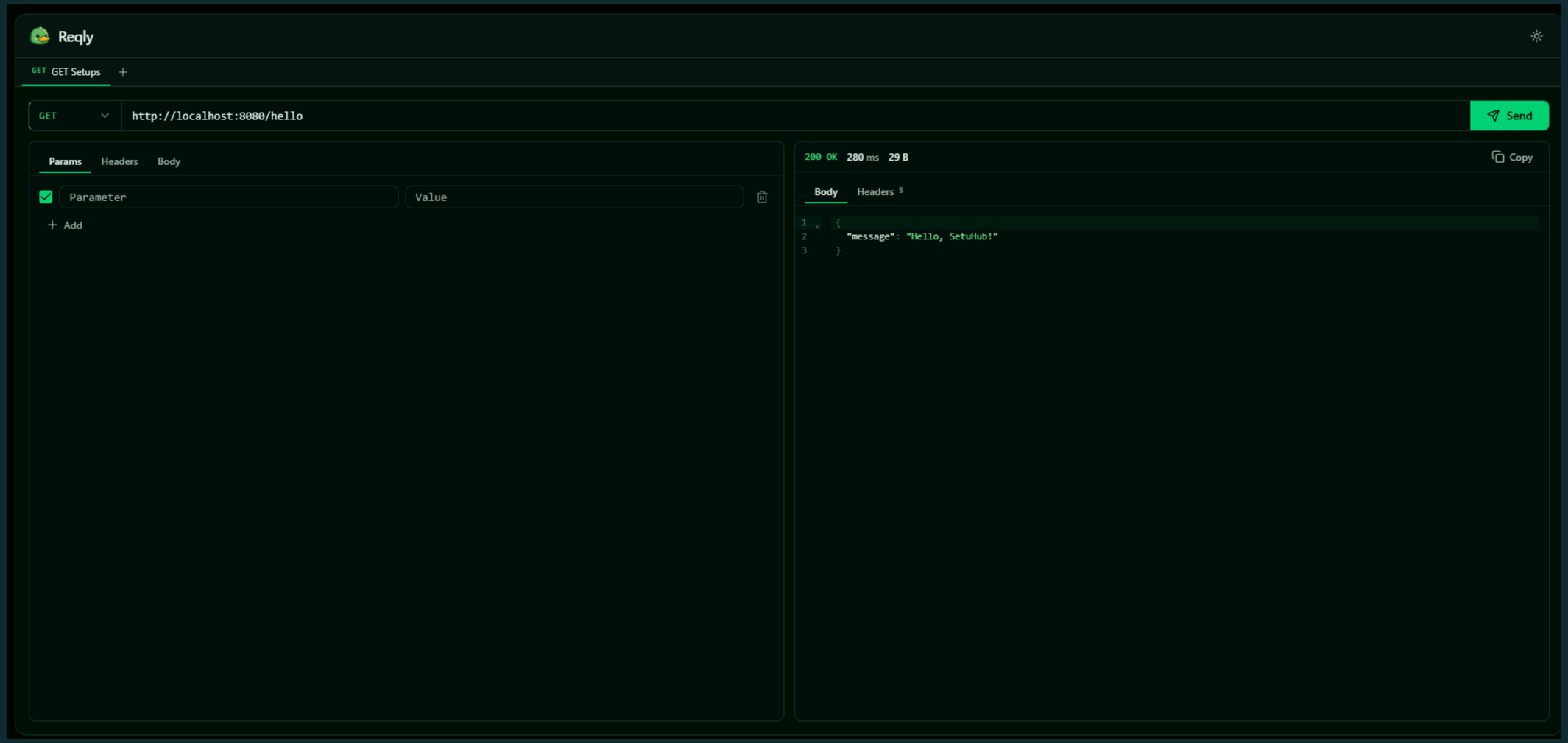

Reqly
GET GET Setups
GET
http://localhost:8080/hello
Send
Params
Headers
Body
Parameter
Value
Add
200 OK 227 ms 16 B
Copy
Body
Headers 5
1 Hello, SetupHub!

Retornando JSON

```
@RestController
public class HelloController {

  @GetMapping("/hello")
  public Map<String, String> hello() {
    return Map.of(
        "message", "Hello, SetuHub!");
  }
}
```

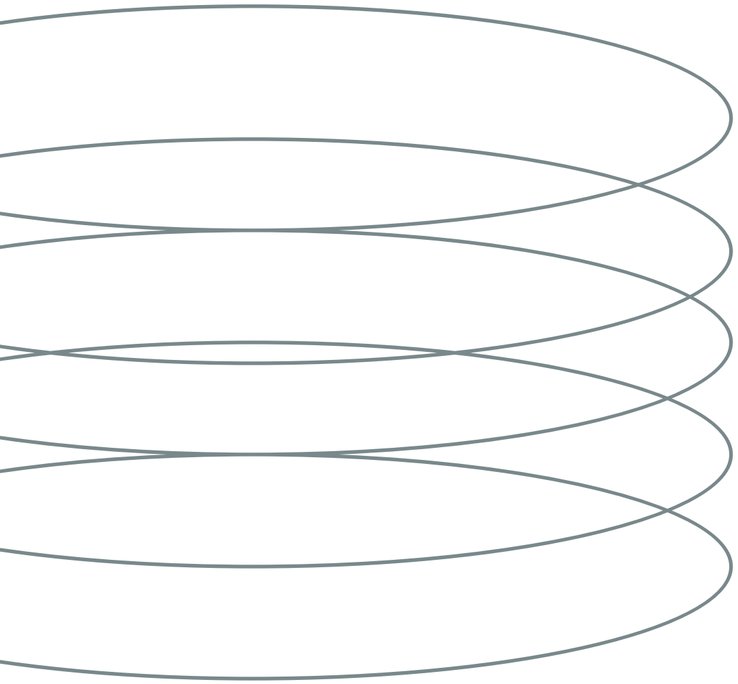



Reqly
GET GET Setups
GET
http://localhost:8080/hello
Send
Params
Headers
Body
Parameter
Value
Add
200 OK 280 ms 29 B
Copy
Body
Headers 5
{
"message": "Hello, SetuHub!"
}

Estrutura do projeto

Vamos começar criando a estrutura de pastas:

config/
controller/
dto/
 ├── *request/*
 ├── *response/*
entity/
exception/
mapper/
repository/
service/
 ├── *impl/*



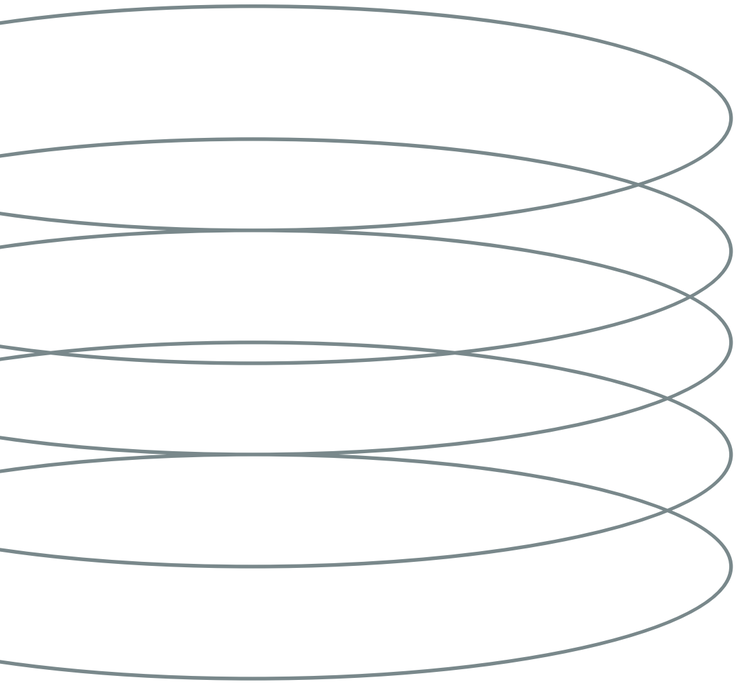
Configurando o CORS

O **CORS** controla quais aplicações podem acessar nossa API pelo navegador. Durante o desenvolvimento, iremos liberar qualquer origem para facilitar os testes com o Reqly e futuras integrações com o front-end.

config/CorsConfig.java

```java
@Configuration
public class CorsConfig implements WebMvcConfigurer {


    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**")
                .allowedOriginPatterns("*")
                .allowedMethods("GET", "POST", "PUT", "DELETE", "PATCH", "OPTIONS")
                .allowedHeaders("*");
    }
}
```



Exceções personalizadas

As **exceções personalizadas** permitem representar erros específicos da aplicação, tornando o código mais organizado e semântico.

exception/GearItemNotFoundException.java

```java
public class GearItemNotFoundException extends RuntimeException {


    public GearItemNotFoundException(UUID id) {
        super("Gear item with ID " + id + " was not found");
    }
}
```

exception/SetupHasItemsException.java

```java
public class SetupHasItemsException extends RuntimeException {

    public SetupHasItemsException(UUID id, long count) {
        super("Cannot delete setup " + id + ": it has " + count + " linked gear item(s). Remove them first.");
    }
}
```

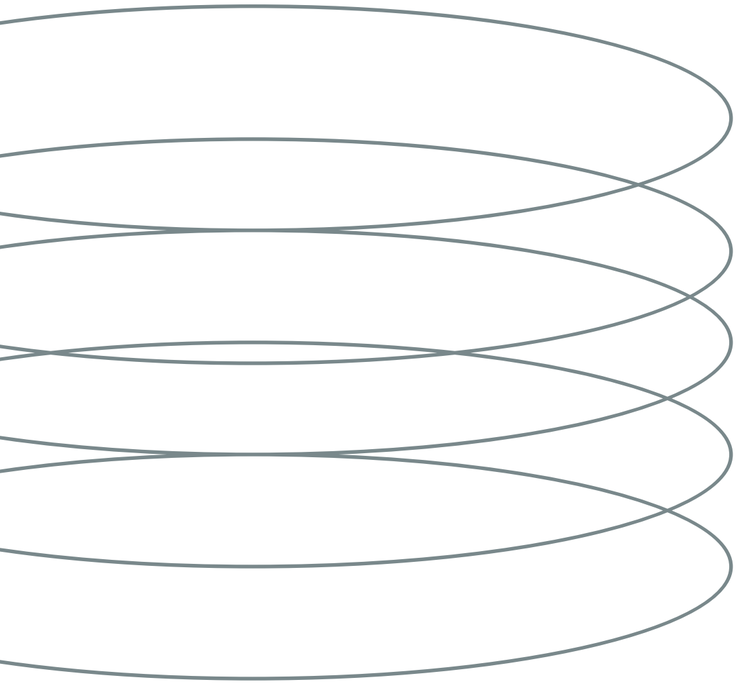
exception/SetupNotFoundException.java

```java
public class SetupNotFoundException extends RuntimeException {

    public SetupNotFoundException(UUID id) {

        super("Setup with ID " + id + " was not found");

    }
}
```



Configurando exceções globais

As **exceções globais** permitem capturar e tratar erros da aplicação em um único lugar, retornando respostas padronizadas e mais organizadas para o cliente.

https://github.com/arturbomtempo-dev/setup-hub/blob/main/server/src/main/java/network/webtech/setuphub_api/exception/GlobalExceptionHandler.java

CRUD de Setup

Estruturando o CRUD de Setup

Criar os arquivos responsáveis pelo CRUD de setups:

- entity/
- enums/
- dto/
- repository/
- mapper/
- service/
- service/impl/
- controller/

Testando o CRUD

Endpoints disponíveis

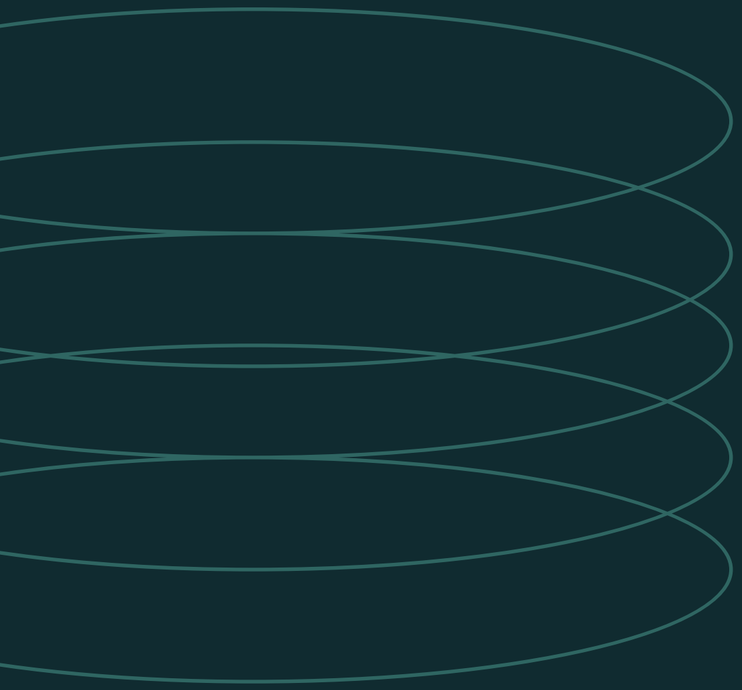
- **GET** → listar setups
- **GET/{id}** → busca setup pelo ID
- **POST** → criar setup
- **PUT** → atualizar setup
- **DELETE** → remover setup

CRUD de itens

Estruturando o CRUD de itens do setup

Criar os arquivos responsáveis pelo CRUD de itens do setup:

- entity/
- enums/
- dto/
- repository/
- mapper/
- service/
- service/impl/
- controller/



Relacionamento entre entidades

Para associar múltiplos itens a um setup, utilizamos um relacionamento entre entidades com **@ManyToOne**.

```
@ManyToOne
@JoinColumn(name = "setup_id", nullable = false)
private Setup setup;
```

Testando o CRUD

Endpoints disponíveis

- **GET** → listar itens
- **GET/{id}** → busca item pelo ID
- **POST** → criar item
- **PUT** → atualizar item
- **DELETE** → remover item

Integração com o front-end

Integração com o front-end

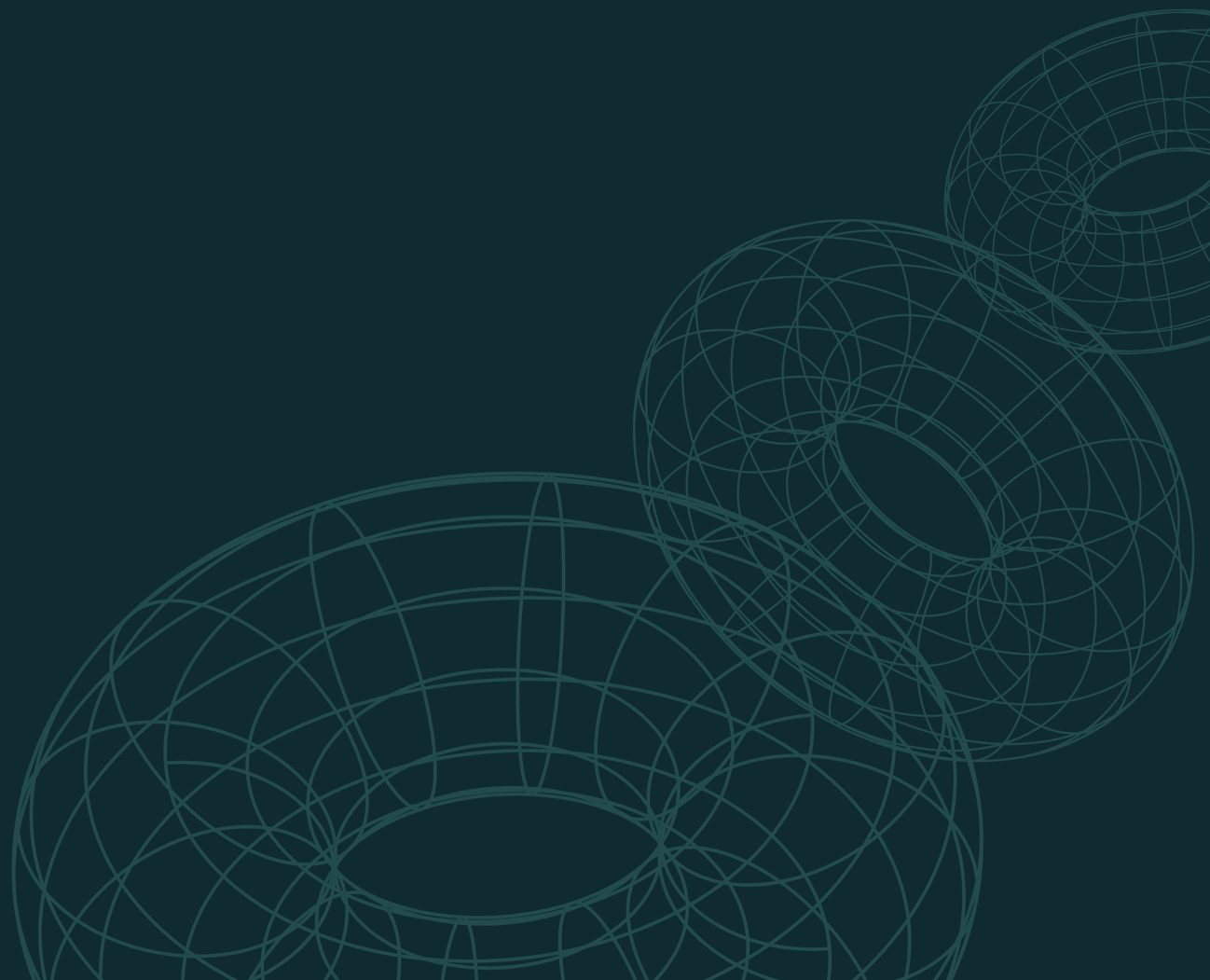
Conectando backend e frontend

A integração com o front-end consiste em consumir os endpoints da API através de requisições HTTP, permitindo comunicação entre a aplicação React e o backend desenvolvido com Spring Boot.

A conexão principal da aplicação já está estruturada. Restam apenas alguns ajustes e implementações marcadas como TODO no código do front-end.

https://github.com/arturbomtempo-dev/setup-hub/tree/main/client

Obrigado.
Dúvidas?



Me acompanhe:
Instagram: @arturbomtempo.dev
YouTube: @ArturBomtempoDev
Linkedin: @artur-bomtempo
GitHub: @arturbomtempo-dev